

SEMINAR TOPIC 06

T06 – API & Contract Testing with Postman + Postbot

Group 04 | Course: Software Testing

Members:

23127047 - Hoang • 23127052 - Huy • 23127104 - Phuong •
23127125 - Thuan • 23127462 - Quang

Why API Testing?

02/16

01

System Backbone

APIs connect Frontend, Backend, and third-party integrations. An API failure paralyzes the entire system immediately.

02

Shift-Left Testing

Detect and isolate bugs early in the service logic layer. Fixing API bugs is far cheaper than fixing them at the GUI level.

03

Speed & Automation

Execute hundreds of test cases in seconds. Easily integrate into CI/CD pipelines for automated regression testing.

CORE REALIZATION:

A critical API bug on the Backend can crash all client platforms (Web, iOS, Android) simultaneously before the UI can handle or display any structured error message.

Topic Scope — T06 Focus Areas

03/16

1. API Functional Testing

Write JavaScript Test Scripts to automate functional validation and business logic checks.

2. API Contract Testing

Validate response structures (JSON Schema) between Consumer and Provider to ensure API contracts are not broken.

3. AI-Augmented Testing

Utilize Postbot AI to auto-generate assertions from actual response structures using natural language prompts.

** Note: Theoretical differences are compared against Pact (dedicated CDC), but hands-on implementation focuses entirely on Postman.*

Why Postman? — Tool Selection Context

04/16

Postman Superiority

Industry Standard: Used by 35M+ users and major software development organizations worldwide.

Visual & Scripted: Excellent UI support for manual requests and powerful automation script runners.

Chai.js Assertion: Built-in JavaScript sandbox with Chai.js assertion library for flexible test suites.

Postbot AI & EShop SUT Context

Postbot AI Assistant: Generates test scripts using natural language prompts, accelerating test writing.

Ideal for EShop SUT: Monolithic Express/Node.js architecture is perfect for lightweight contract testing.

Cost-Effective: Free tier provides all core testing features and 50 AI credits per month.

Key Features Overview

05/16

1. Request Builder & Auth

Build REST requests (GET, POST, PUT, DELETE).
Inherit Authorization (Bearer Token) at the
Collection level for centralized security.

2. Variable Chaining

Program dynamic API workflows: Extract JWT tokens
from Login response and dynamically inject them as
headers in subsequent requests.

3. Collection Runner

Run complete integration workflows directly from
Postman's GUI, supporting Data-Driven Testing
(DDT) with external datasets.

4. Newman CLI & Reporter

Execute test suites headless from terminal. Export
rich reports (HTML, JUnit XML) to integrate into
CI/CD pipelines.

Basic API Requests & Status Codes

06/16

GET

Read/Query data
Status: 200 / 404

POST

Create new resources
Status: 200 / 201

PUT

Update existing data
Status: 200

DELETE

Remove resources
Status: 200 / 204



AUTOMATED AUTHENTICATION TIP:

Instead of configuring Bearer Tokens for individual requests, set it at the Collection level. Through Auth Inheritance, all child requests inherit the authorization headers automatically.

Common Authentication Mechanisms

1. Bearer Token (JWT):

Token generated by the server and sent in the Authorization Header. Standard for SPA and Mobile apps.

2. API Key:

Static key passed via Header or Query Parameter.

3. Basic Auth:

Base64 encoded username:password sent directly.

4. Session Cookie:

Server maintains session state; browser sends cookies automatically.

EShop SUT Automation Workflow

Step 1: POST /api/login

Client sends valid user credentials to the Server.

Step 2: Extract & Auto-save

Post-response script extracts JWT token and saves it in {{auth_token}}.

Step 3: Automatic Inheritance

Subsequent protected requests use the environment variable automatically.

Scripting in Postman (Pre-request vs. Post-response)

08/16

Pre-request Scripts (Before sending)

- **Purpose:** Prepare dynamic data or calculate inputs before the request is sent.
- **Real-world Example:** Generating a random email during `POST /api/register` to avoid 'Email already exists' errors:

```
const randomEmail =  
`test_${Math.floor(Math.random() *  
10000)}@eshop.com`;  
pm.environment.set("email", randomEmail);
```

Then use `{{email}}` directly in the Request Body JSON.

Post-response Scripts / Tests (After receiving)

- **Purpose:** Validate API response quality (assertions) and extract variables for workflow chaining.
- **Variable Chaining:**
Extract login token and user ID to pass them to GET Cart, POST Cart, and Checkout endpoints.

```
const responseData = pm.response.json();  
pm.test("Save Auth Token & User ID", function  
( ) {  
    pm.environment.set("auth_token",  
        responseData.token);  
    pm.environment.set("user_id",  
        responseData.user.id);  
});
```

** This will be demonstrated live on Postman during the demo phase.*

Contract Testing with Postman (API Interface Validation)

09/16

JSON Schema Validation

Postman integrates the Ajv Validator (v6.12.5) to support JSON Schema draft-07.

Asserts key data types (`integer`, `string`), required fields, and format constraints (`format: email`).

Structure Assertions

Validates other aspects of the API contract:

- Status Code: Checks if the correct HTTP status is returned.
- Headers: Verifies the response format (e.g., `application/json`).
- SLA Response: Ensures the API meets performance limits (<800ms).

Living Contracts

Save sample responses as Examples directly in Postman.

These Examples serve as the API documentation, which can spin up Postman Mock Servers for Frontend/Backend decoupled development.

Collection Runner & Newman CLI (Automated Run)

10/16

Collection Runner (GUI) & DDT

- **Automated Workflow Execution:** Run the entire test folder sequentially, configuring iterations and delays to simulate real user behavior.
- **Data-Driven Testing (DDT):** Instead of hardcoding credentials, feed an external CSV/JSON file to run tests across multiple accounts and verify success/error paths.

Newman CLI (Headless Terminal Run)

- **Headless Execution:** Newman runs exported collections and environments directly from terminal:

```
newman run EShop_Collection.json \  
-e EShop_Env.json -r cli,html
```

- **Reporting Formats:** Generate HTML reports for visual inspection or JUnit XML reports for CI/CD dashboard integration (e.g., Jenkins, GitHub Actions).

Postbot AI in Automated Test Generation

11/16

Postbot AI Interaction Flow

01 Send the request to get an actual sample response.

02 Go to the Tests tab and click 'Ask Postbot'.

03 Type a natural language query (e.g., 'Test for response').

04 Review the generated JavaScript assertions and save.

Custom Prompting Power

In default mode, Postbot only checks basic types. To test business logic, write specific calculations:

Prompt sent to Postbot:

*"Write a test that calculates the expected discount as $\text{total_amount} * 10 / 100$ for a percent coupon, and asserts `discount_amount` matches this calculation."*

With custom guidance, Postbot writes logical code instead of simple type checks.

Limitations of Postbot AI

12/16

LIMITS

Happy-Path Focus

Postbot is heavily biased towards generating successful (status 200/201) tests. It completely misses negative error paths and edge cases where business logic bugs usually reside.

LIMITS

Sample Dependency

AI works by analyzing the active sample response. If the request has not been run, or if the server is down, Postbot cannot generate assertions.

LIMITS

No Business Logic Concept

By default, AI only checks surface-level structure (strings, numbers). It cannot verify numerical correctness or stateful business rules without custom prompts.

Failure Modes — 3 Ways Tools Mislead You

13/16

FM1

Anchoring on Buggy Response

Postbot 'learns' and locks in server bugs instead of catching them. If the sample response has wrong calculations (e.g., negative discount ``-4,500,000d``), Postbot writes tests validating this bug as correct behavior.

FM2

Superficial Error Testing

AI-generated error tests only check if the ``error`` property is a string, completely ignoring whether the error message matches the actual business context or stateful limits.

FM3

Runner Environment Mismatch

Collection Runner's environment selector is separate from the main workspace. Mismatched active environments cause variable resolution failures, which the Runner hides.

Postman vs. Pact Comparison (Contract Testing)

14/16

Comparison Aspect	Postman (Lightweight Contract)	Pact (Dedicated CDC Framework)
Target Architecture	Monolith (EShop SUT), small to medium codebases	Highly distributed Microservices architecture
Core Mechanism	JSON Schema validation within post-response scripts	Contracts (.json) published to Pact Broker
Setup & Infrastructure	Extremely simple, no infrastructure required	Complex, requires Pact Broker host & mock frameworks
Learning Curve	Low, uses standard JavaScript/Chai.js scripts	High, requires understanding consumer-driven contract model
Verdict for SUT	OPTIMAL CHOICE (Fits EShop monolith scale)	Overkill (Not needed for monolithic codebase)

* Verdict: Postman delivers quick interface validation at minimum cost for monolithic or independent development teams.

E2E Integration Flow (Success Path)

Goal: Demonstrate variable chaining and flow execution via Collection Runner:

1. Authentication (POST /api/login)
 - ↳ Extracts and auto-saves token to `{{auth_token}}`
2. View Product Details (GET /api/products/:id)
 - ↳ Extracts product ID dynamically
3. Add to Cart (POST /api/cart)
4. Checkout (POST /api/checkout)

-> Verifies that all endpoints return matching schemas.

Bugs Captured via Negative Contract

Goal: Expose interface contract violations on EShop SUT using edge cases:

- Request: Retrieve nonexistent product
`GET /api/products/999`
- Expected Contract Behavior:
Status code `404 Not Found` with a structured error envelope.
- SUT Bug Detected:
Returns `200 OK` with an empty response body, representing a broken contract that could crash client apps.

Key Takeaways & Q&A

16/16

Use Postman Right

Go beyond manual API calls.
Leverage pre-request/post-response scripts, runners, and Newman for complete automation.

AI Limitations

AI (Postbot) is great for boilerplate, but human testers are decisive for edge cases, business rules, and negative tests.

API Contracts

Contract tests prevent client-side crashes by validating data shapes between Frontend and Backend.

THANK YOU FOR LISTENING!

Moving to Live Demo on Postman...

Youtube Link: <https://youtu.be/EHq4QRIPkGk?si=99hrkHml-ct2ELI8>